

Project 2

NUMN21/FMNN25: Advanced Numerical Algorithms in Python
Robert Klöfkorn and Andreas Langer

This project is devoted to artificial neural networks.
This assignment has 8 tasks.

Application: Recognizing handwritten numbers

504192

Most people effortlessly recognize these digits as 504192. This ease is deceptive. The difficulty of visual pattern recognition becomes obvious when trying to write a computer program to recognize digits like the above. What seems easy when we do it ourselves suddenly becomes extremely difficult. Simple notions about how we recognize shapes - “a 9 has a loop at the top and a vertical line at the bottom right” - turn out to be not so easy to express algorithmically. If you try to specify such rules, you quickly get lost in a quagmire of exceptions, restrictions and special cases. It seems hopeless.

Neural networks approach the problem differently. The idea is to use a large number of handwritten digits, called training examples, and then develop a system that can learn from these training examples. In other words, the neural network uses the examples to automatically derive rules for recognizing handwritten digits. In addition, by increasing the number of training examples, the network can learn more about handwriting and thus improve its accuracy.

In your implementation, libraries or modules that trivialize the task (such as `TensorFlow` or `PyTorch`) are not allowed.

Task 1

Implement a feedforward neural network (as a class) consisting of 3 layers (input, hidden, output layer), where each layer can contain any number of neurons. Use the sigmoid function as the activation function.

Task 2

Reading in MNIST data (provided in canvas). The data is separated into training data (50 000), validation data (10 000), and test data (10 000).

Task 3

Implement the stochastic gradient method (SGD) to train the network. The implementation of the SGD should allow for different mini-batch sizes and different numbers of epochs. An epoch is the complete pass of the training data through the learning algorithm.

Task 4

Implement the backpropagation algorithm (used in SGD to effectively calculate the derivative).

Task 5

Train and test the accuracy of the network for the following parameters:

- Input layer with $784 + 1$ neurons
- hidden layer with $30 + 1$ neurons
- Output layer with 10 neurons

As loss function use the quadratic function (square loss)

$$\frac{1}{2n} \sum_x \|h_w(x) - y\|_2^2,$$

where (x, y) is a pair of training data, n the amount of used training data, and h_w represents the neural network.

Task 6

Print an output of the learning success per epoch.

Task 7

Implement an attack on the trained neural network.

Task 8

In this task, investigate how closely a neural network can fit a small training set.

Use the first 50 samples of the MNIST training data provided in Task 2. Consider a fully connected feedforward neural network with one hidden layer, 784 input neurons, and 10 output neurons. You may choose the number of neurons in the hidden layer, the activation function, and the training procedure.

Use the quadratic loss from Task 5,

$$\frac{1}{2n} \sum_{(x,y)} \|h_w(x) - y\|_2^2,$$

with $n = 50$.

Try to reduce the training loss as much as possible. Report the number of hidden neurons, the activation function, the training procedure, and the smallest loss you obtain. You are encouraged to modify the training procedure if this helps to reduce the loss. In particular, you may experiment with different mini-batch sizes, learning rates, and learning-rate strategies during training.

The loss must be evaluated directly on the raw output of the neural network. No rounding, thresholding, `argmax`, or other post-processing of the output is allowed before evaluating the loss.

Competition. Among all networks that achieve a training loss smaller than 10^{-8} , the winner is the network using the smallest number of neurons in the hidden layer.

In case of a tie, the winner is the network that reaches a training loss below 10^{-8} in the smallest number of epochs. The number of epochs is counted up to the first epoch at which the training loss falls below the threshold.

If there is still a tie, the network with the smaller final training loss wins.

Prize. The winning group will be named as the record holder for this task on the Canvas course page. In addition, the winning group only needs to complete 6 of the 12 tasks in Project 3. The six tasks may be chosen freely.

Lycka till!